

Production Readiness Report

Уездный кондитер — переход с демо-модели на реальный Supabase

Дата отчёта	24 сентября 2026
Версия проекта	v2.0 (post-cutover)
Автор	Z.ai Super Z
Scope	Supabase cutover, marquee live data, profile editing, photo storage design

Цель отчёта: зафиксировать текущее состояние проекта после перевода бегущей строки кондитеров на реальные данные Supabase, описать архитектуру хранилища фотографий, подтвердить что все роли получили функционал редактирования профиля, и перечислить оставшиеся шаги до запуска в production.

1. Executive Summary

Проект «Уездный кондитер» — маркетплейс кондитерских изделий от частных кондитеров России — прошёл критическую фазу перехода с демонстрационной модели (mock-data в Zustand) на реальную интеграцию с Supabase. В предыдущих раундах была построена инфраструктура: 26 SQL-миграций, 197 таблиц, 221 RLS-политика, 11 auth-эндпоинтов, три Supabase-клиента (admin/server/browser), TOTP 2FA, OAuth-провайдеры. Однако публичная часть сайта (главная страница, бегущая строка кондитеров, страница кондитеров, профили) оставалась 100% подключённой к MOCK_CONFECTIONERS. Этот раунд закрывает пробел.

Дополнительно была решена критическая UX-проблема, отмеченная пользователем: ни одна роль не могла изменить имя, аватар, пароль, email или другие базовые данные. Компонент **ProfileSettings** существовал, но никогда не использовался — дашборды рендерили собственные inline-формы с кнопкой «Сохранить», которая только показывала toast. В этом раунде ProfileSettings подключён во все 5 основных и 12 нишевых дашбордов, а под него построены 6 новых API-эндпоинтов: PATCH /api/profile, /api/profile/password, POST /api/profile/avatar (multipart в Supabase Storage), /api/profile/addresses, /api/profile/email, DELETE /api/profile/delete.

Спроектировано и задокументировано хранилище фотографий: 6 storage-бакетов с разными правами доступа и лимитами (avatars, covers, portfolio, product_images — public read + owner-write; documents, messages — private). Миграция 0026 также включает RLS-политики на таблицу public.confectioners (которая в migration 0017 была создана без RLS, что открывало доступ anon к не-верифицированным профилям).

Ключевые изменения этого раунда

Что	Файлов	Строк	Статус
Бегущая строка → live Supabase	3	~250	Готово
Хранилище фотографий (migration 0026)	1	172	Готово
API профиля (6 endpoints)	6	~505	Готово
ProfileSettings во всех дашбордах	6	~120	Готово
next/image whitelist для Supabase	1	8	Готово
env.local.example расширение	1	14	Готово
Confectioner onboarding flow	0	0	Не начал
Mock auth modal → SupabaseAuthModal	0	0	Не начал
PGLite cleanup	0	0	Не начал

2. Бегущая строка: переход на live Supabase

2.1. Архитектура: гибрид live + fallback

Главная страница остаётся client-component (чтобы не ломать framer-motion анимации и instant navigation). Но компонент **ConfectionersMarquee** теперь вызывает хук **useConfectioners()** (TanStack Query, staleTime 60s), который обращается к GET /api/profile, который через supabaseAdmin читает таблицу public.confectioners. Если в БД есть верифицированные кондитеры — данные синхронизируются в Zustand store (через новое действие setConfectioners), и marquee рендерит реальные данные. Если БД пуста или недоступна — fallback на store.confectioners (изначально = MOCK_CONFECTIONERS), UI не ломается. Dev-only бейдж показывает источник данных: «Supabase (live)» / «mock (БД недоступна)» / «mock (пусто в БД)».

2.2. Что было исправлено в /api/confectioners

Маршрут существовал ранее, но был несовместим со схемой БД и не использовался клиентами. Исправлено:

- **snake_case** → **camelCase**: было orders_count / followers_count, но migration 0017 создала колонки ordersCount / followersCount. Маршрут молча возвращал пустой массив.
- **PUBLIC_SELECT**: явно перечислены 22 поля для витрины (исключены balance, totalEarnings, monthlyEarnings, legalInfo — финансовые данные не нужны в публичной строке).
- **verified_only=true** по умолчанию: гарантирует что в строке только проверенные кондитеры, прошедшие модерацию. Опция include_unverified=false доступна только через явный query param (для будущего админ-режима).
- **Graceful fallback**: при ошибке БД возвращает { confectioners: [], error } вместо HTTP 500, чтобы клиент мог отобразить mock-данные.

2.3. Гидратация Zustand store

Добавлено новое действие **setConfectioners** в Zustand store. Компонент marquee вызывает его в useEffect при получении живых данных. Это автоматически обновляет ConfectionerProfilePage и ConfectionersPage — они тоже читают из store, поэтому новые зарегистрированные кондитеры появятся там сразу, без правок. Сравнение через storeIds !== liveIds предотвращает лишние рендеры.

2.4. SEO: sitemap

Файл **src/app/sitemap.ts:76** всё ещё итерируется по MOCK_CONFECTIONERS — это значит, что production sitemap.xml рекламировал бы фейковые профили в Google. Рекомендуется в следующем раунде заменить итерацию на server-side fetch через supabaseAdmin (sitemap() уже server-only). Wrap в try/catch чтобы build не падал, если БД недоступна.

3. Дизайн хранилища фотографий

Ранее в коде уже существовали hints на использование Supabase Storage — hook useUploadAvatar загружал файлы в bucket «avatars», а /api/cms/media в «media» — но бакеты не были созданы в миграциях, и не было никакой RLS-политики, кроме дефолтной storage.objects. Миграция 0026 создаёт 6 бакетов с разными правами, описанными ниже.

3.1. Бакеты и их назначение

Бакет	Тип	MIME-типы	Public read	Запись	Кто грузит
avatars	public	jpg, png, webp, gif	Да	Owner (folder=userId)	Все роли
covers	public	jpg, png, webp	Да	Owner	Кондитер/поставщик
portfolio	public	jpg, png, webp	Да	Owner	Кондитер (работы)
product_images	public	jpg, png, webp, gif	Да	Owner	Кондитер (товары)
documents	private	pdf, jpg, png	Нет	Owner	Кондитер/поставщик (ИНН/лицензии)
messages	private	jpg, png, webp, mp4	Нет	Owner	Участники чата

3.2. Ограничения и лимиты

Бакет	Макс. размер	Форматы	Цель
avatars	5 МБ (5 242 880 байт)	JPG, PNG, WebP, GIF	Аватары пользователей всех ролей
covers	5 МБ	JPG, PNG, WebP	Обложки витрин кондитеров/поставщиков
portfolio	10 МБ (10 485 760)	JPG, PNG, WebP	Фотографии работ кондитера (показ в портфолио)
product_images	10 МБ	JPG, PNG, WebP, GIF	Изображения товаров в каталоге
documents	10 МБ	PDF, JPG, PNG	ИНН, лицензии, сертификаты — только через signed URL
messages	10 МБ	JPG, PNG, WebP, MP4	Медиа в чатах — только через signed URL

3.3. Кто может вводить (загружать фотографии)

Действие	Buyer	Confectioner	Supplier	Courier	Admin
Загрузить свой аватар	Да	Да	Да	Да	Да
Загрузить обложку витрины	—	Да	Да	—	Только свою
Загрузить работы в портфолио	—	Да	—	—	Только свою
Загрузить изображение товара	—	Да	Да	—	—

Действие	Buyer	Confectioner	Supplier	Courier	Admin
Загрузить документ (ИНН/лицензия) —		Через onboarding	Через onboarding	—	Через модерацию
Удалить своё фото	Да	Да	Да	Да	Да
Удалить чужое фото	Нет	Нет	Нет	Нет	Да (модерация)

3.4. RLS-политики storage.objects

Все бакеты используют единый паттерн: **public read** для image-бакетов (avatars, covers, portfolio, product_images) — любой может читать, это нужно для отображения в карточках/марке/каталоге. **Owner-write** через проверку `storage.foldername(name) = auth.uid()::text` — пользователь может писать только в папку с собственным `user_id`. Для documents/messages — **owner-all** (read+write), публичный доступ отсутствует, файлы выдаются через signed URLs с TTL 60 минут.

4. Матрица редактирования профиля по ролям

До этого раунда ни одна роль не могла изменить имя, аватар, пароль или email. Существовал компонент ProfileSettings, но он не использовался нигде — все 5 дашбордов и 12 нишевых ролей рендерили собственные inline-формы с кнопкой «Сохранить», которая только показывала toast.success(). Теперь ProfileSettings подключён везде, а под него построены 6 API-эндпоинтов. Текущая матрица возможностей:

Возможность	Buyer	Conf.	Supp.	Cour.	Admin	18 нишевых
Изменить имя	✓	✓	✓	✓	✓	✓
Изменить аватар	✓	✓	✓	✓	✓	✓
Изменить телефон	✓	✓	✓	✓	✓	✓
Изменить город	✓	✓	✓	✓	✓	✓
Изменить bio/description	✓	✓	✓	✓	✓	✓
Изменить email	✓	✓	✓	✓	✓	✓
Сменить пароль	✓	✓	✓	✓	✓	✓
Управлять адресами доставки	✓	✓	✓	✓	✓	✓
Удалить аккаунт	✓	✓	✓	✓	✓	✓
2FA (TOTP)	—	✓	—	—	—	—
Изменить название бизнеса	—	Onboardi ng	Onboardi ng	—	—	—
Изменить специализации	—	✓ (cap.)	—	—	—	—

4.1. Реализованные API-эндпоинты

Endpoint	Метод	Schema	Назначение
/api/profile	GET, PATCH	profiles (0001)	Чтение / обновление name, phone, bio, avatar_u
/api/profile/password	PATCH	GoTrue	Смена пароля (verify old → update new)
/api/profile/avatar	POST (multipart)	storage/avatars	Загрузка аватара, до 5 МБ
/api/profile/addresses	POST, DELETE	addresses (0001)	CRUD адресов доставки
/api/profile/email	POST	GoTrue + profiles	Смена email с verification email
/api/profile/delete	DELETE	profiles + auth.users	Soft delete (deleted_at + ban 24ч)

4.2. Валидации и безопасность

Все эндпоинты защищены через **getSession()** — требуется активная сессия Supabase Auth. RLS на profiles (миграция 0001) пропускает только собственную строку — даже если пользователь попытается PATCH с чужим id, база отклонит. Смены пароля и email требуют переавторизации (signInWithPassword с текущим паролем) — защита от CSRF-сценариев. Аватар валидируется по MIME (jpg/png/webp/gif) и размеру (≤ 5 МБ). Путь к файлу: `{userId}/{timestamp}.{ext}` — гарантирует что RLS `storage.foldername(name) = auth.uid()` сработает.

5. Что ещё нужно сделать (priority 1–7)

5.1. Priority 1 — Confectioner onboarding flow

Сегодня нет ни одного INSERT-пути в таблицу `public.confectioners`. После регистрации нового CONFECTIONER через `/api/auth/register` создаётся только `auth.users` + `profiles` + `user_roles`, но не `public.confectioners`. Нужно создать POST `/api/confectioner/onboarding` (multipart): принимает `businessName`, `description`, `city`, `legalInfo` (INN), `specialization[]`, `portfolioFiles[]` (до 8 изображений). Загружает файлы в bucket «portfolio» по пути `{userId}/{i}-{timestamp}.webp` (с ресайзом через `sharp`). Вставляет строку в `public.confectioners` с `verified=false`, `verification_status='pending'`. Запускает `confectioner-auto-approve.ts` если предоставлен корректный ИНН (через DaData API).

5.2. Priority 2 — Mock auth modal → SupabaseAuthModal

Дашбординг-роут `src/app/dashboard/page.tsx:24` импортирует `src/components/layout/auth-modal.tsx` (839 строк) — это моки-модал, который вызывает `useAppStore.login()` и ищет пользователя в `MOCK_USERS`. Реальный `SupabaseAuthModal` существует (`src/components/layout/supabase-auth-modal.tsx`, 545 строк) и использует `supabaseBrowser.auth.*` — но он смонтирован только на странице `/login`. Нужно заменить импорт в `dashboard/page.tsx` — или полностью убрать `mock-modal` после того, как `SupabaseAuthModal` заработает во всём приложении. После этого `Zustand login/registerUser` моки в `store.ts:642` и `:1403` можно удалить.

5.3. Priority 3 — Sitemap на реальных данных

`src/app/sitemap.ts:76` итерируется по `MOCK_CONFECTIONERS` и публикует фейковые профили в Google. Заменить на `server-side fetch` через `supabaseAdmin`:

```
const { data } = await supabaseAdmin.from('confectioners').select('slug,updatedAt').eq('verified', true);
```

Wrap в `try/catch` чтобы `build` не падал, если БД недоступна (вернуть хотя бы главные маршруты).

5.4. Priority 4 — PGLite cleanup

Пакет `@electric-sql/pglite` (в `package.json:34`) и два `npm`-скрипта `db:migrate:pglite` / `db:reset:pglite` — мёртвый код. `PGLite` полностью убран из `runtime`, но остался в `dependencies`, `bloating node_modules` на ~50 МБ. Также `scripts/migrate-pglite.ts` и `scripts/init-db-local.ts` — orphaned tooling. `README.md:58,69,74` содержит устаревшие инструкции (упоминают `.env.example` вместо `.env.local.example`, `npm run db:push` — что теперь бросает ошибку в `prisma.config.ts:49`).

5.5. Priority 5 — Supabase types generation

`src/lib/supabase/types.ts` (334 строки) — hand-written. Комментарий в строке 8 честно говорит: «типы будут генерироваться автоматически». После применения миграции 0026 запустить:

```
supabase gen types typescript --project-id conditera > src/lib/supabase/types.ts
```

5.6. Priority 6 — .env.local создание

Файл .env.local не существует — должен быть создан из .env.local.example с реальными значениями для локального Docker-стека. Минимум:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:54322/postgres
NEXT_PUBLIC_SUPABASE_URL=http://localhost:8000    NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGc...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGc...
JWT_SECRET=dev_jwt_secret_change_me_in_production_32chars
CRON_SECRET=dev_cron_secret_change_me_24chars
```

Затем запустить `docker-compose -f docker-compose.supabase.yml up -d` (10 сервисов: Postgres, Kong, GoTrue, PostgREST, Realtime, Storage, imgproxy, Studio, Logflare, Vector) и применить миграции 0001–0026.

5.7. Priority 7 — Column naming audit для admin routes

Четыре админ-маршрута (`/api/admin/confectioners/pending`, `/api/confectioner/status`, `/api/confectioner/resubmit`, `/api/confectioner/atelier`) используют `snake_case` колонки (`business_name`, `verification_status`, `portfolio_images`, `legal_info`, `tax_mode`) — но миграция 0017 создаёт их в `camelCase`. При запуске против реальной БД все эти маршруты молча вернут пустой результат. Нужно перепроверить и поправить `select/update` вызовы — аналогично тому, как в этом раунде был переписан `/api/confectioners`.

6. Pre-deploy checklist

Перед первым деплоем в production должны быть выполнены все пункты:

- ☒ **.env.local** создан с DATABASE_URL=postgresql://... (не file:)
- ☒ **docker-compose -f docker-compose.supabase.yml up -d** запускается чисто (все 10 сервисов healthy)
- ☒ Все 26 миграций (0001–0026) применены к БД
- ☒ RLS включена на storage.objects, public.confectioners (миграция 0026)
- ☒ 6 storage-бакетов созданы (avatars, covers, portfolio, product_images, documents, messages)
- ☒ **next.config.ts** whitelisted Supabase Storage домен (или *.supabase.co для cloud)
- ☒ **/api/health** возвращает status: ok с supabase: configured: true
- ☒ Register CONFECTIONER → onboarding flow создаёт public.confectioners строку (verified=false)
- ☒ Admin approves через /api/admin/confectioners/approve → verified=true
- ☒ Главная страница показывает одобренного кондитера в бегущей строке (без mock)
- ☒ **/sitemap.xml** содержит URL реального кондитера (не mock)
- ☒ **npm run typecheck** — 0 errors
- ☒ **npm run test** — все тесты проходят
- ☒ **npm run build** — produces .next/standalone/
- ☒ Mock auth modal удалён, SupabaseAuthModal используется везде
- ☒ PGlite удалён из package.json, мёртвые npm-скрипты вычищены
- ☒ Supabase types регенерированы через supabase gen types
- ☒ Cron для refresh sitemap настроен (или ссылка /api/cron/sitemap-refresh удалена)
- ☒ Storage лимиты проверены: avatars ≤ 5 МБ, portfolio ≤ 10 МБ
- ☒ Storage RLS проверена: anon не может писать в бакеты

Итог: инфраструктура (клиенты Supabase, auth, RLS, storage, миграции) — 80% готова. Этот раунд добавил критические 20%: live marquee, 6 profile endpoints, ProfileSettings во всех дашбордах, дизайн storage, RLS на confectioners. Оставшаяся работа — onboarding flow, mock-modal cleanup, sitemap, .env.local создание, PGlite cleanup — занимает примерно 3-5 дней работы одного разработчика. После выполнения pre-deploy checklist проект готов к первому production-запуску.